

THE ENTERPRISE PR REVIEW PLAYBOOK

Volume IV — Agentic AI Review Architecture

Designing the Autonomous Review Pipeline — Specialist Reviewer Agents,
MCP/A2A Orchestration, Policy Engines, and Governance Boards

A reference architecture for enterprise agentic review, grounded in the current state of the MCP and A2A protocol ecosystem, LangGraph-style multi-agent orchestration patterns, and established governance-board practice adapted for autonomous review pipelines.

Series: 5 Volumes · Volume 4 of 5

July 2026

Contents

Section 15 — Agentic AI PR Review

- 15.1 Design principles for an autonomous review pipeline
- 15.2 Fourteen specialist reviewer agents

Section 16 — Multi-Agent Review Workflow

- 16.1 The protocol layer: MCP and A2A
- 16.2 Reference architecture
- 16.3 Orchestration, voting, and conflict resolution
- 16.4 Confidence scoring and evidence collection
- 16.5 Human approval gates
- 16.6 Traceability, audit logs, and the policy engine

Section 17 — Enterprise Review Governance

- 17.1 Change Advisory Boards and Architecture Review Boards
- 17.2 Production and Operational Readiness Reviews
- 17.3 How governance boards integrate with agentic review

About This Series

Section 15 — Agentic AI PR Review

An agentic review pipeline is not "Volume 3's tools, but more of them." The distinction that matters: Volume 3 covered tools that *analyze* a diff and report findings to a human. This section designs a pipeline where specialist agents can independently investigate, cross-reference each other's findings, reach a collective judgment, and — within tightly bounded limits — take action, with the governance structures (Section 16-17) that keep that autonomy safe.

15.1 Design Principles for an Autonomous Review Pipeline

- **Specialization over generality** — a single do-everything reviewer agent underperforms a set of narrowly-scoped agents each optimized for one concern, mirroring why human review works better as role-based review (Volume 1, Section 2) than as one generalist reading everything equally.
- **Advisory by default, authoritative by exception** — every agent in this section defaults to producing findings for human or higher-authority-agent review; only a small, explicitly enumerated set of low-risk, high-confidence, reversible actions should ever be auto-applied without a human checkpoint (see Section 16.5).
- **Bounded iteration** — every agent has a hard cap on tool calls, iterations, and cost per review, enforced by the orchestrator, not left to the agent's own judgment — the direct governance fix for the infinite-loop and context-explosion anti-patterns from Volume 1, Section 12.3.
- **Evidence over assertion** — every finding a review agent produces must be traceable to a specific piece of evidence (a line of code, a test result, a policy document) that a human or downstream agent can independently verify, rather than an unsupported claim.
- **Composable, not monolithic** — agents communicate through the standard protocol layer (Section 16.1) rather than tightly-coupled custom integration code, so a reviewer agent can be swapped, upgraded, or added without redesigning the pipeline.

15.2 Fourteen Specialist Reviewer Agents

Each agent below maps to a domain from Volumes 1–2 and is specified using a consistent template: inputs, context, tools/knowledge sources, memory, policies, decision criteria, output, confidence score, and escalation path. This specification format is deliberately the contract each agent exposes to the orchestrator (Section 16), regardless of which underlying model or framework implements it.

Architecture Reviewer Agent

Inputs	PR diff, linked ADR/RFC (if any), service dependency graph.
Context	Repository structure, module boundary definitions, DDD bounded-context map.
Knowledge sources / Tools	Static dependency analyzer, ADR repository search (MCP resource), architecture linting rules.
Memory	Prior architectural decisions for this service (long-term); current PR's own iteration history (short-term).
Policies	Enterprise reference architecture; layering rules (no domain-layer imports from infra-layer).
Decision criteria	Does the change respect module boundaries and, if architecturally significant, is it backed by an ADR? Flags, does not block.
Output	Structured finding list with severity, each citing the specific dependency-graph edge or file violated.
Confidence score	High only when a boundary violation is unambiguous per the dependency graph; low for judgment calls about degree of coupling.
Escalation path	Any finding below a configured confidence threshold, or any finding contradicting an existing Accepted ADR, escalates to a human architect rather than being auto-resolved.

Security Reviewer Agent

Inputs	PR diff, dependency manifest changes, SBOM diff.
Context	OWASP Top 10:2025 rule set, prior security findings for this repository, known CVE database.
Knowledge sources / Tools	SAST engine, secret scanner, dependency/SCA scanner (OSV.dev, NVD), SBOM generator.
Memory	Repository-specific accepted-risk exceptions (to avoid re-flagging a previously reviewed and accepted tradeoff every single run).
Policies	No hardcoded secrets; no wildcard IAM; every new dependency pinned and from an allow-listed registry (Volume 3, Section 14.5).
Decision criteria	Does this change introduce a new OWASP Top 10:2025 category finding, weaken an existing control, or add unvetted supply-chain risk?
Output	Findings tagged by OWASP category and severity, each with a proof-of-concept trace where feasible.
Confidence score	High for pattern-matched, deterministic findings (hardcoded secret, wildcard IAM); lower for context-dependent authorization-logic findings.
Escalation path	Any Critical/High finding blocks auto-merge eligibility and requires a human security reviewer (Volume 2, Section 4) regardless of confidence score.

Performance Reviewer Agent

Inputs	PR diff, historical performance baseline for touched code paths, load-test results if available.
---------------	--

Context	Known hot paths in the service; existing latency/throughput SLOs.
Knowledge sources / Tools	Static complexity analysis, query-plan analysis for DB-touching changes, benchmark harness.
Memory	Historical performance regressions previously introduced by similar changes in this repository.
Policies	No new N+1 query patterns; no unbounded loops over externally-controlled input size.
Decision criteria	Does this change plausibly regress a latency- or throughput-sensitive path?
Output	Findings with an estimated impact magnitude where determinable (e.g., "adds one DB round-trip per loop iteration").
Confidence score	Moderate — performance impact is often only fully knowable under real load, so this agent should rarely claim high confidence without an actual benchmark run.
Escalation path	Any finding on a path with an existing SLO escalates to the owning team's on-call or SRE rather than being silently logged.

Platform Reviewer Agent

Inputs	IaC diff (Terraform/Helm/K8s manifests), CI/CD pipeline config diff.
Context	Golden-path deployment patterns; current resource quota and cost baseline per environment.
Knowledge sources / Tools	`terraform plan` execution, Kubernetes manifest linter, cost-estimation tool (e.g., Infracost-style).
Memory	Prior infra incidents traced to a specific pattern in this repository.
Policies	Every workload has resource limits; every third-party CI action pinned by commit SHA (Volume 2, Section 5.3).
Decision criteria	Does this change deploy safely, stay within cost guardrails, and follow the golden path?
Output	Findings plus the actual `plan` diff for any destroy-and-recreate operation flagged as high-risk.
Confidence score	High for deterministic policy violations (missing limits, unpinned action); lower for cost projection accuracy.
Escalation path	Any destroy-and-recreate operation on a production-tagged resource escalates unconditionally to a human, regardless of confidence (direct mitigation for the incident class in Volume 3, Section 14.6).

Documentation Reviewer Agent

Inputs	PR diff, existing README/runbook/ADR content for touched areas.
Context	Whether this PR changes a public interface, a common workflow, or an on-call-relevant behavior.
Knowledge sources / Tools	Doc-freshness checker (flags docs referencing changed function signatures or config keys).
Memory	None required beyond the current PR and the current state of the docs it touches.
Policies	PR template's "Architecture Impact" checkbox must be answered, not left blank.
Decision criteria	Does this change make existing documentation inaccurate, or does it lack documentation a reviewer would expect (Volume 2, Section 8)?
Output	A list of specific doc files likely now stale, with the specific line or section in question.
Confidence score	High — doc staleness is comparatively easy to detect mechanically (signature/config-key mismatch).
Escalation path	Never blocks; always advisory, since documentation debt is real but rarely urgent enough to justify auto-blocking a merge.

API Reviewer Agent

Inputs	OpenAPI/protobuf/GraphQL schema diff.
Context	Existing consumer list (from a service catalog, where available); prior API versioning history.
Knowledge sources / Tools	Schema-diff tool (breaking-change detector), consumer-driven contract test results.
Memory	Prior breaking changes and how they were handled for this API.
Policies	No breaking change without an explicit version bump and deprecation window (Volume 2, Section 7).
Decision criteria	Is this schema change additive, or does it break an existing contract a consumer depends on?
Output	Pass/fail on breaking-change detection, with the specific field/type change cited.
Confidence score	High — schema compatibility is a largely deterministic, mechanically checkable property.
Escalation path	Any detected breaking change without a corresponding version bump blocks auto-merge eligibility unconditionally.

Data Reviewer Agent

Inputs	Database migration diff, schema-registry compatibility check result.
Context	Table size and traffic volume for touched tables; existing partitioning/replication topology.
Knowledge sources / Tools	Migration dry-run against a production-scale snapshot; schema-registry compatibility-mode check.
Memory	Prior migrations against this table and their observed lock duration/impact.

Policies	Expand/contract pattern required for anything not purely additive (Volume 2, Section 6).
Decision criteria	Is this migration reversible, non-locking at production scale, and backward-compatible during a rolling deploy?
Output	Findings with estimated lock duration and row-count-scaled runtime where determinable.
Confidence score	Moderate to high depending on whether a production-scale dry run was actually executed versus estimated.
Escalation path	Any migration estimated to hold a lock beyond a configured threshold, or any non-reversible migration, escalates to a human DBA/data engineer before merge.

Testing Reviewer Agent

Inputs	PR diff, existing test suite, coverage report diff.
Context	Which code paths in the diff are exercised by existing tests versus newly added ones.
Knowledge sources / Tools	Coverage instrumentation, mutation-testing tool (optional, for confidence in existing tests' strength), flaky-test history.
Memory	Historical flakiness record per test, to avoid re-flagging a known-flaky test as a new regression.
Policies	New logic requires a corresponding test exercising both the happy path and at least one failure path (Volume 1, Section 2.9).
Decision criteria	Does this change have adequate test coverage of its new or modified behavior, particularly failure paths?
Output	Coverage delta plus specific untested branches, cited by file and line.
Confidence score	High for coverage-delta measurement; lower for judging whether coverage is meaningfully testing behavior versus just executing lines.
Escalation path	Advisory only by default; can be configured to block merge below an org-defined coverage floor.

Compliance Reviewer Agent

Inputs	PR diff, data-classification tags on touched fields, applicable regulatory scope (e.g., PCI, HIPAA, GDPR) for the service.
Context	Current compliance posture and any open remediation items for this service.
Knowledge sources / Tools	Data-classification scanner, policy-as-code engine (e.g., OPA/Rego-style rule evaluation).
Memory	Prior compliance findings and their resolution for this repository.
Policies	PII/regulated data must be encrypted, access-logged, and retained per its classification's policy.
Decision criteria	Does this change touch regulated data in a way that violates an applicable compliance policy?
Output	Findings mapped to the specific regulatory clause or internal policy violated.
Confidence score	High for policy-as-code deterministic checks; lower for novel data-handling patterns not covered by existing rules.
Escalation path	Any finding touching regulated data escalates to a human compliance officer; this agent never has auto-approval authority.

Observability Reviewer Agent

Inputs	PR diff, existing instrumentation coverage for the touched service.
Context	Standard telemetry conventions (metric naming, trace-span conventions) for the organization.
Knowledge sources / Tools	Instrumentation-coverage linter, dashboard/alert-config diff checker.
Memory	None beyond current repository state.
Policies	Every new externally-facing endpoint emits standard request/latency/error metrics and trace spans by default (Volume 1, Section 2.7–2.8).
Decision criteria	Will this change be visible in existing dashboards and alerting, or does it introduce an observability blind spot?
Output	Findings listing specific missing instrumentation, mapped to the org's standard telemetry schema.
Confidence score	High — instrumentation presence/absence is mechanically checkable against a known convention.
Escalation path	Advisory only; does not block, but is weighted heavily in the SRE/platform composite score (Section 16.3).

AI Safety Reviewer Agent

Inputs	PR diff for any prompt, agent configuration, or tool-permission change.
Context	Existing eval suite results for touched prompts/agents; prior incident history for this agent.

Knowledge sources / Tools	Eval harness execution, prompt-injection test suite, tool-scope diff analyzer.
Memory	Prior AI-safety findings and their resolution for this agent/prompt.
Policies	No consequential/destructive tool granted without a human-approval gate (Volume 3, Section 14.6); hard iteration/cost caps required on every agent loop.
Decision criteria	Does this change expand an agent's tool scope, remove a safety check, or lack eval coverage for a prompt modification?
Output	Findings citing the specific eval regression, missing gate, or scope expansion.
Confidence score	High for structural findings (missing iteration cap, missing approval gate on a destructive tool); lower for eval-score regression severity judgment.
Escalation path	Any removal of an existing human-approval gate, or any new destructive tool grant, blocks auto-merge unconditionally and escalates to the AI Architect role (Volume 1, Section 2.12).

Cost Reviewer Agent

Inputs	IaC diff, autoscaling configuration diff, projected traffic change if stated in the PR.
Context	Current spend baseline per service/environment; org-wide cost anomaly thresholds.
Knowledge sources / Tools	Cost-estimation tool, autoscaling-ceiling checker.
Memory	Historical cost anomalies traced to specific infra patterns.
Policies	Every autoscaling configuration has an explicit, bounded maximum (Volume 2, Section 5.4).
Decision criteria	Does this change introduce unbounded cost exposure (missing autoscale ceiling, oversized default resource request)?
Output	Projected monthly cost delta with the specific configuration driving it.
Confidence score	Moderate — cost projections are estimates, not guarantees, and should be presented with that caveat.
Escalation path	Any missing autoscale ceiling escalates to platform engineering; large projected cost deltas escalate to the requesting team's engineering manager.

Governance Reviewer Agent

Inputs	Full set of findings from all other agents for this PR.
Context	Organization's review policy configuration (which findings block, which are advisory, per repo/team).
Knowledge sources / Tools	Policy engine (evaluates the aggregate finding set against configured rules).
Memory	None beyond the current PR's aggregate finding set.
Policies	The org's codified merge policy — e.g., "any Critical Security finding blocks; any two or more High findings from different agents requires a second human approval."

Decision criteria	Given the aggregate finding set, does this PR meet the organization's configured bar for auto-eligible merge, or does it require human review, and at what level?
Output	A single routing decision: auto-eligible, standard human review, or escalated review (with the triggering findings cited).
Confidence score	This agent does not have its own confidence score — it is a deterministic policy evaluator over other agents' outputs, not a judgment-forming agent itself.
Escalation path	By definition, this agent's output <i>is</i> the escalation routing for the whole pipeline; it has no further escalation path of its own beyond the configured governance board (Section 17).

Enterprise Standards Reviewer Agent

Inputs	PR diff, applicable enterprise reference architecture and coding-standard documents.
Context	Which enterprise standards apply to this repository/domain (not every repo is in scope for every standard).
Knowledge sources / Tools	Enterprise standards document search (MCP resource), pattern-conformance linter.
Memory	Prior standards exceptions granted for this repository, to avoid re-flagging an already-approved deviation.
Policies	Conformance to published enterprise patterns, or an explicit, time-boxed, owned exception on record (Volume 1, Section 2.5).
Decision criteria	Does this change conform to an existing enterprise standard, or does it introduce an undocumented deviation?
Output	Findings citing the specific standard and the specific deviation.
Confidence score	High for deterministic pattern-matching against documented standards; lower for judgment calls about whether a standard actually applies to this case.
Escalation path	Any undocumented deviation from a mandatory (not advisory) enterprise standard escalates to the Enterprise Architecture review board (Section 17.1).

Section 16 — Multi-Agent Review Workflow

16.1 The Protocol Layer: MCP and A2A

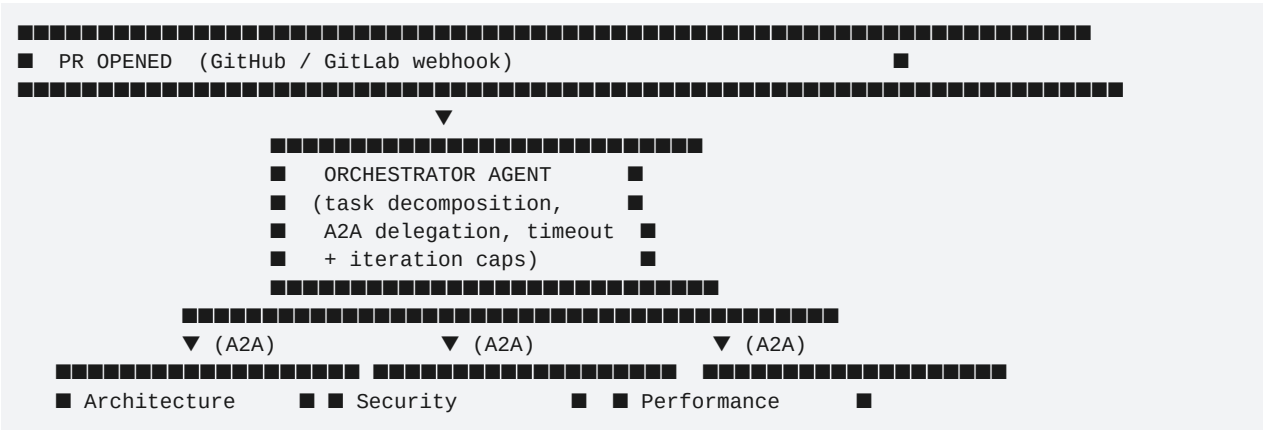
By mid-2026 the protocol landscape for exactly this kind of multi-agent system has consolidated around two complementary, non-competing standards, both now under open, vendor-neutral governance.

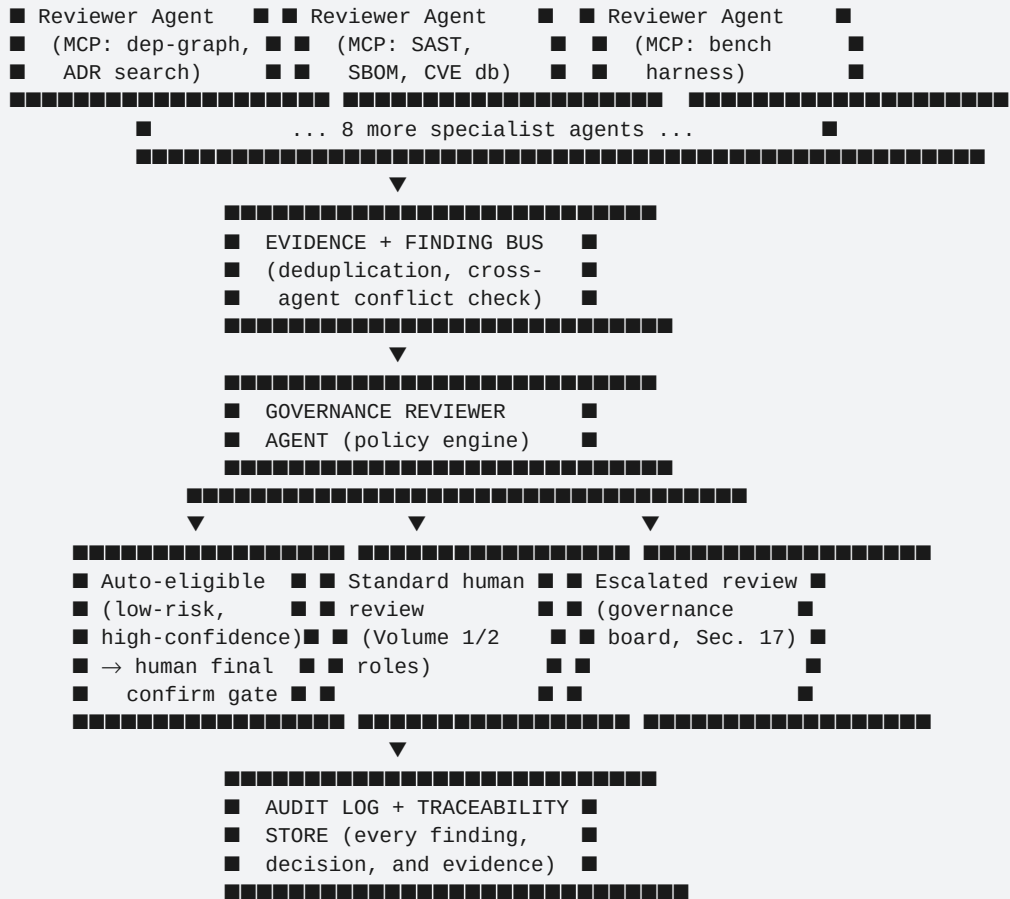
The Two-Layer Protocol Stack

Protocol	Origin	What It Solves	Role in This Pipeline
MCP (Model Context Protocol)	Anthropic, released Nov. 2024	How a single agent accesses tools and data — a reviewer agent's connection to its SAST scanner, its ADR repository, its SBOM generator. Reported at roughly 97 million downloads and adopted across Anthropic, OpenAI, Google, and Microsoft tooling by early 2026.	Every specialist agent's individual toolset in this pipeline (Section 15.2) is exposed via MCP.
A2A (Agent2Agent)	Google, released April 2025; donated to Linux Foundation mid-2025	How independent agents — potentially built by different teams or vendors — discover each other's capabilities, hand off tasks, and coordinate, using structured "Agent Cards" for capability discovery and defined task-lifecycle states (submitted, working, completed, failed). Reported production-grade enterprise adoption and 150+ supporting organizations by April 2026.	The orchestrator agent's delegation to each specialist reviewer agent, and any cross-agent finding hand-off (e.g., the Architecture Reviewer flagging something for the Security Reviewer to examine further), runs over A2A.

The mental model that recurs consistently across current practitioner guidance: **MCP is how an agent uses its tools; A2A is how agents work with each other.** Neither protocol, on its own, answers who is allowed to do what or how an organization knows when something went wrong — that is the governance layer this section and Section 17 build on top of the protocol layer.

16.2 Reference Architecture





Rendered as Mermaid source below; paste into any Mermaid-compatible renderer (GitHub Markdown, mermaid.live, or a docs site) to view as a diagram. boxes represent A2A-addressable agents; arrows into each box represent its MCP tool connections, omitted here for width.

16.3 Orchestration, Voting, and Conflict Resolution

- **Parallel dispatch, bounded timeout** — the orchestrator dispatches to all applicable specialist agents in parallel (not sequentially), with a hard per-agent timeout; an agent that doesn't return within its budget is treated as "no finding, flagged as incomplete" rather than blocking the whole pipeline indefinitely.
- **Reviewer voting on overlapping findings** — when two agents flag the same code region for different reasons (e.g., Performance and Data both flag the same migration), the finding bus surfaces both rather than deduplicating them away, since they represent genuinely independent concerns even when co-located.
- **Conflict resolution when agents disagree** — if the Architecture Reviewer approves a pattern the Enterprise Standards Reviewer flags as a deviation, this is not silently resolved by either agent; it routes to a human at the intersection of both domains (typically the Enterprise Architect role, Volume 1 Section 2.5), since the disagreement itself is the signal, not noise to be averaged away.
- **Weighted composite scoring** — the Governance Reviewer Agent's policy engine, not any individual specialist, computes the final routing decision, using configured weights per finding severity and agent domain rather than a simple majority vote — a single Critical Security finding should outweigh five Low Documentation findings, and the policy engine encodes that explicitly rather than leaving it to an implicit voting scheme.

16.4 Confidence Scoring and Evidence Collection

Every specialist agent's finding carries a confidence score, but — following the verification-pass pattern already emerging in production tools (Volume 3, Section 14.4) — confidence is not self-reported by the finding-generating agent alone. A separate verification step re-checks each candidate finding against actual code behavior (running the referenced test, re-evaluating the referenced policy rule, re-executing the referenced query plan) before the finding is finalized, specifically to counter the hallucination risk documented in Volume 3, Section 14.5. Every finding in the evidence store carries: the specific artifact it cites (file:line, policy clause, CVE ID), the tool or check that produced it, and the verification step's pass/fail result — nothing reaches a human or the policy engine as a bare assertion.

16.5 Human Approval Gates

This is the single most important governance mechanism in the entire pipeline, and the direct structural fix for the incident pattern described in Volume 3, Section 14.6 (an agent taking a destructive action without human sign-off).

- **Gates are enforced in the permission model, not the prompt.** An agent should be technically incapable of executing a gated action — the credential or tool scope simply isn't available to it — rather than merely instructed not to. A prompt-level instruction is a suggestion; a missing credential is a guarantee.
- **The gated-action list is explicit and enumerated, not inferred.** Destructive infra operations, database migrations against production, merging a PR, granting a new tool/permission to another agent, and anything touching regulated data are the baseline set; individual organizations extend this list but should never shrink it via an agent's own judgment.
- **Auto-eligible PRs still pass through a lightweight human confirmation**, not a fully unattended merge — the reference architecture in 16.2 shows even the "auto-eligible" path terminating in a human final-confirm step, reflecting that no tool surveyed in Volume 3 recommends or implements fully unattended merging in production use today.

- **Escalation is a first-class output, not a failure mode.** An agent routing a decision to a human is the system working as designed, not the system failing to be autonomous enough — measuring "percentage auto-resolved" as a success metric in isolation creates exactly the pressure that erodes this safeguard over time.

16.6 Traceability, Audit Logs, and the Policy Engine

- Every finding, every agent decision, every human override, and every escalation is written to an append-only audit log — the agentic-review analog of the ADR's append-only, superseded-not-edited discipline from Volume 2, Section 9.3.
- The policy engine (the Governance Reviewer Agent's rule set) is itself version-controlled and reviewed with the same rigor as production code — a change to "what blocks a merge" is at least as consequential as the code changes it's gating, and should go through this same review discipline.
- Audit logs should be queryable for after-the-fact analysis: which agent's findings correlate with actual production incidents, which agents produce the highest false-positive rate, and which escalation paths are triggered most often — the data that lets an organization tune the pipeline the same way vendors tune their tools using the thumbs-up/down signal described in Volume 3.

Section 17 — Enterprise Review Governance

17.1 Change Advisory Boards and Architecture Review Boards

Governance Boards and Their Relationship to Agentic Review

Board	Function	Integration Point
Change Advisory Board (CAB)	Reviews and approves high-risk production changes, typically at a cadence (weekly, or on-demand for urgent changes) separate from individual PR review — the governance layer above PR review, not a replacement for it.	Escalated-review PRs from the reference architecture (16.2) that trip a defined blast-radius or risk threshold route here rather than merging on standard PR approval alone.
Architecture Review Board (ARB)	Reviews architecturally significant decisions — typically RFC-stage, before implementation (Volume 2, Section 10) — and adjudicates the Enterprise Standards / Architecture Reviewer agents' escalations described in Section 16.3.	Agent-flagged undocumented deviations from mandatory enterprise standards, and disagreements between specialist agents at an architectural boundary, land here.
Security Review Board	Adjudicates accepted-risk exceptions for security findings that a team wants to ship despite a flagged issue — the human authority the Security Reviewer Agent's unconditional escalation (Section 15.2) ultimately reports to.	Any Critical/High security finding a team wants to override rather than fix goes through this board, not a single engineer's judgment call.
Production Readiness Review (PRR)	A one-time gate before a new service first reaches production, checking the full breadth of Volume 1–2 concerns (ownership, on-call readiness, SLOs, rollback, security posture) in one sitting rather than piecemeal across many PRs.	The Platform, SRE, Security, and Observability Reviewer agents' outputs can be aggregated into a PRR checklist automatically, though the review itself remains a human sign-off event.
Operational Readiness Review (ORR)	A recurring or pre-major-change version of the PRR, re-validating operational readiness (updated runbooks, current on-call rotation, load-tested capacity) before a significant change or seasonal traffic event.	Similar aggregation role as the PRR, run periodically rather than once.

17.2 Production and Operational Readiness Reviews

These are the highest-altitude review gates in the whole pipeline, and — deliberately — the ones least amenable to full automation. A PRR is checking whether an organization, not just a codebase, is ready: is there a named owner for 2am pages, does the on-call rotation actually have runbook access, has the team actually load-tested at expected scale rather than just configuring an autoscaler and hoping. Agentic review agents can pre-populate the checklist (which reduces the human meeting from an hour of discovery to minutes of confirmation) but the human sign-off itself should not be automated away, since a PRR's actual function is organizational accountability, not just technical verification.

17.3 How Governance Boards Integrate with Agentic Review

The design principle that ties Sections 15–17 together: **agentic review agents feed governance boards; they do not replace them.** An agent can pre-screen, aggregate evidence, flag conflicts, and route escalations with far more consistency and speed than a human doing the same triage manually — but the actual authority to accept risk, override a finding, or certify organizational readiness remains with a named human or board, exactly the same non-negotiable boundary Volume 3, Section 14.7 established for individual review tools, now extended to the full multi-agent pipeline. The audit trail from Section 16.6 is what makes this workable at scale: a governance board reviewing an escalated PR sees the full evidence chain an agent assembled, rather than starting its own investigation from zero.

About This Series

This is Volume 4 of a five-part Enterprise PR Review Playbook. Volume 1 covers workflow, ownership models, and role-based human review playbooks. Volume 2 covers deep domain review. Volume 3 covers AI-assisted review tools available today and their benchmarked strengths and failure modes. Volume 5 collects case studies, master checklists, review scorecards, and a review maturity model that spans all four preceding volumes.

Generated as a synthesized practitioner reference. The agent-protocol ecosystem (MCP, A2A, and adjacent standards) is evolving rapidly; adoption figures and governance details reflect publicly reported status as of mid-2026 and should be re-verified against current protocol documentation before architecture decisions are finalized.